

一、ROS 配置与实操实验指导

1. 实验描述

本次课为本门课程的第一次实验，通过前面教材知识点的学习，我们对基于 ROS 的机器人开发中的一些指令、名词有一个大致的了解。本次实训，我们要通过实际操作来掌握具体的相关技能。

首先要熟悉 ROS 机器人开发中软件环境配置以及机器人与本地 PC 进行远程连接的方法，配置和连接完成后即可在本地 PC 查看和编辑相关的工程文件，通过 C++或 Python 进行代码编程或修改。

其次要掌握基于 ROS 系统的商服机器人的相关操作，包括创建工作空间、调用相关功能包，会运行节点、查看话题等。

通过以上内容学习，在后续进行复杂实验时就具备了相应的操作基础，理解后续实验的相关原理并进行实验验证时操作方能得心应手。

2. 实验目标

(1) 安装与配置软件环境，掌握个人 PC 远程连接商服机器人进行工程文件查看、代码编辑的基本方法；

(2) 查看工程文件，了解商服机器人的文件系统结构，熟悉各实验工程；

(3) 掌握商服机器人 ROS 系统的相关基础操作，包括创建工作空间、调用所需的功能包、运行节点、查看话题等。

3. 实验步骤

以下包括了商服机器人使用和配置的全部操作步骤，学习时可根据以下过程依次理解并操作。

步骤 1：启动机器人，基于机器人进行基础操作

确保商服机器人在自身有电或者充电状态下，按下机器人底座顶部后侧的电源按钮即可开机，然后等待机器人自行开机进入系统桌面。

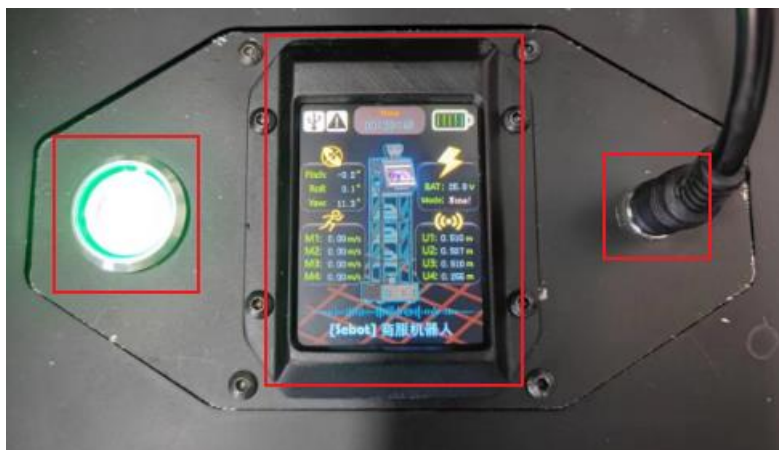


图 1-1 电源开关、状态显示屏与充电接口

系统开机后，机器人上方后侧有 USB 接口用于插入无线键鼠和遥控手柄的接收器，同时还有网线接口用于远程连接，进行键鼠/手柄控制或远程连接时需要先插入接收器或网线。

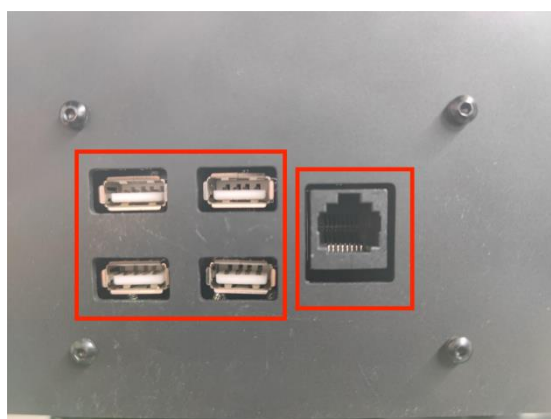
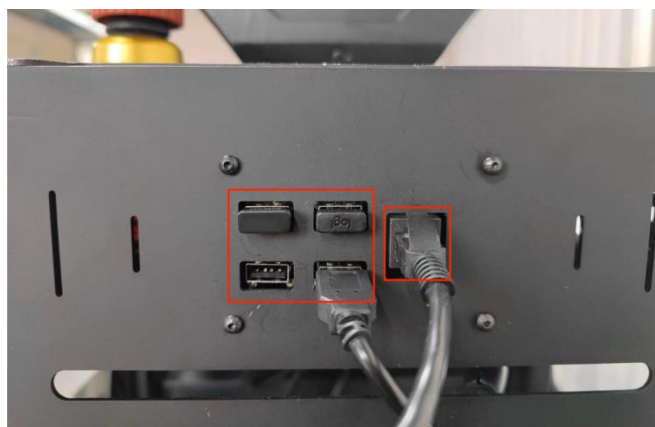


图 1-2 USB 接口和网线接口

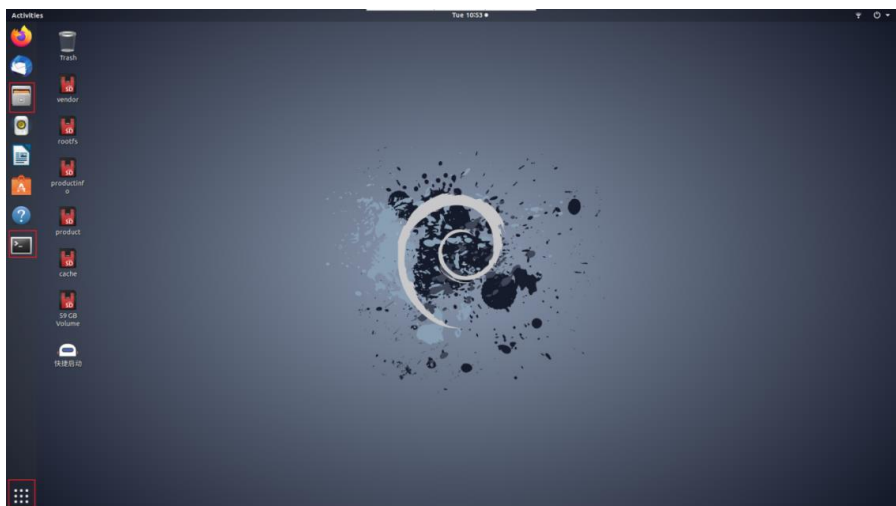


图 1-3 系统桌面

该桌面为 Linux 系统常规操作界面，在该操作系统中，点击左侧菜单栏的应用程序即可打开指定的功能，如果没有对应的应用程序，可以点击左下方的按钮显示所有的应用程序（如上图示）。应用程序中最为常用的是“Files”和“Terminal”，“Files”即文件夹，可以打开根目录，从而查看整个商服机器人的工程文件夹，“Terminal”即终端，可以输入指令执行指定的操作，如下图示。

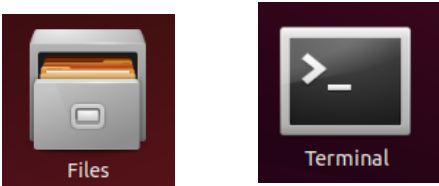


图 1-4 文件夹图标(左)与终端图标(右)

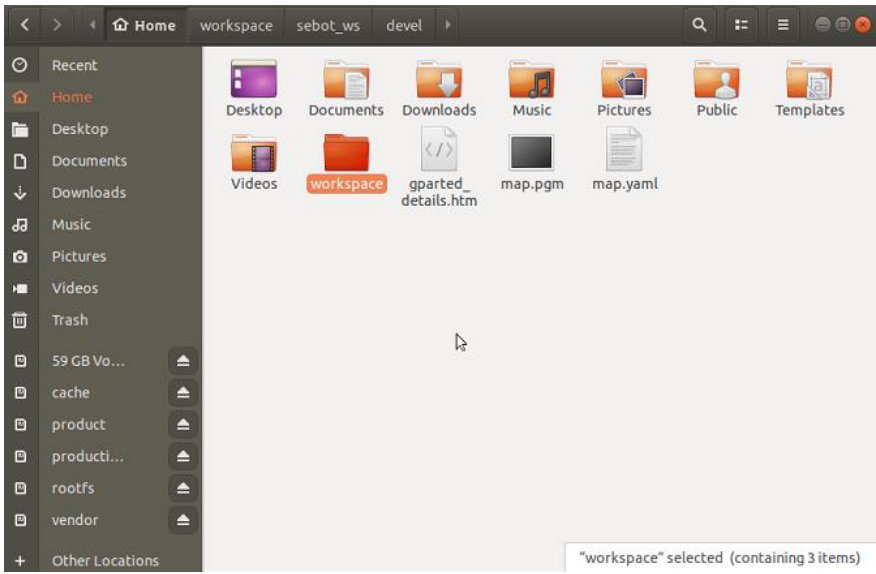


图 1-5 打开文件夹 Files 的界面（商服机器人（机械臂版）界面略有不同）

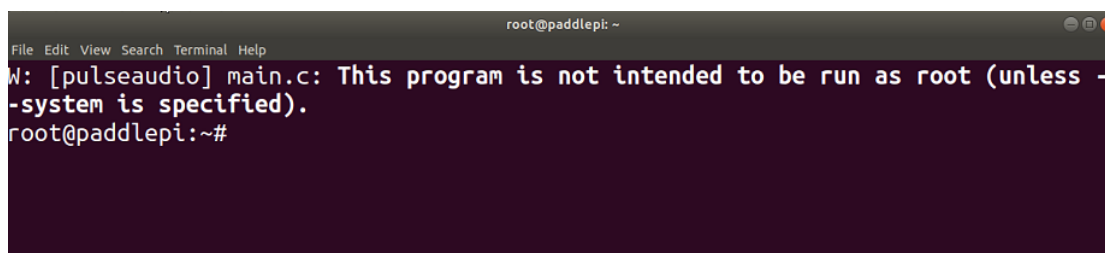


图 1-6 打开终端 Terminal 的界面

步骤 2：个人 PC 的软件安装及远程连接

机器人学习包含了编程部分，本商服机器人可基于 Visual Studio Code 进行 C++编程或 Python 编程，在机器人上已内置了 VSCode 软件，可直接在机器人上进行编程或工程代码文件的修改、优化。但编程学习也可以在个人 PC 上进行，所以在个人 PC 上也可以自行安装 VSCode，通过远程连接的方式对机器人上的工程文件进行修改、调整或优化。所以本步骤主要包含 Visual Studio Code 编译软件安装、个人 PC 与商服机器人的远程连接两部分。

(1) Visual Studio Code 编译工具安装

第一步，在个人电脑上打开“商服机器人安装支持包”->“Visual Studio Code 安装包”。



商服机器人安装支持包



Visual Studio Code 安装包

图 1-7 编译工具

第二步，双击 VSCodeUserSetup-x64-1.84.2 进行安装，安装时根据需要选择安装路径，然后执行默认设置进行安装操作，安装完成后重启计算机。

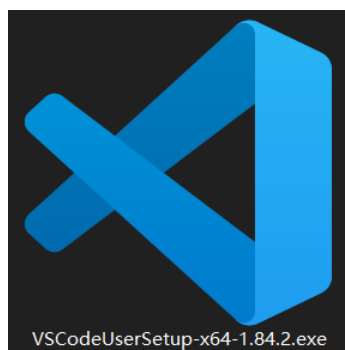


图 1-8 Visual Studio Code 安装文件

（2）配置以太网

将个人 PC 与商服机器人进行远程连接前，需要先将电脑与机器人使用网线连接，然后配置个人 PC 上的以太网。

第一步，打开个人 PC 上的控制面板，选择“网络和 Internet”选项；



图 1-9 选择“网络和 Internet”

第二步，选择“网络和共享中心”；



图 1-10 选择“网络和共享中心”

第三步，点击“以太网”选项，具体名称以实际显示为准；



图 1-11 选择“以太网”

第四步，进行以太网 ipv4 的设置。

按照下图从左到右的顺序，依次选择“属性”-“Internet 协议版本 4 (TCP/IPv4)”-“属性”，然后设置如图所示的 IP 地址，完成以太网 ipv4 的设置。

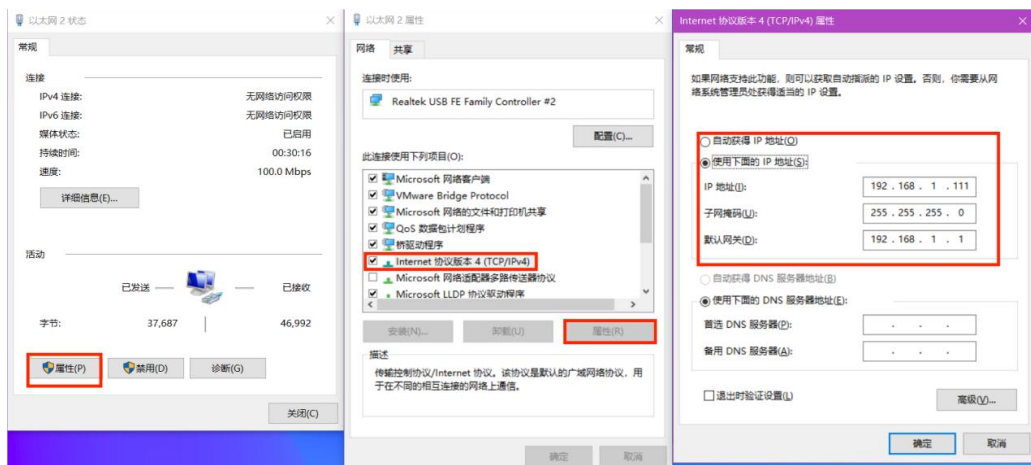


图 1-12 配置以太网

(3) 远程共享文件夹

第一步，上述以太网设置完成后，进入个人 PC 的“此电脑”，在网络位置下方空白处单击右键，选择“添加一个网络位置”；

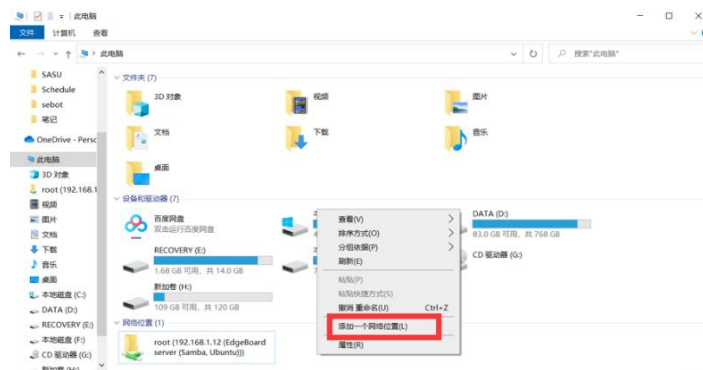


图 1-13 添加网络位置

第二步，点击两次“下一步”，直到出现输入框，然后按图所示输入对应的 IP 地址：
\\192.168.1.254\root\root\workspace，点击“下一步”后给此地址设置一个名称，设置完成后点击“下一步”确定即可。

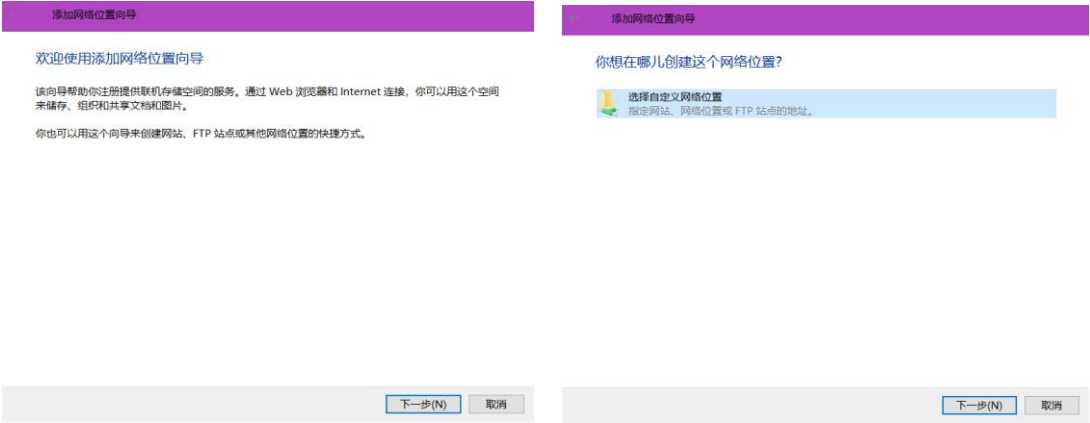


图 1-14 添加位置

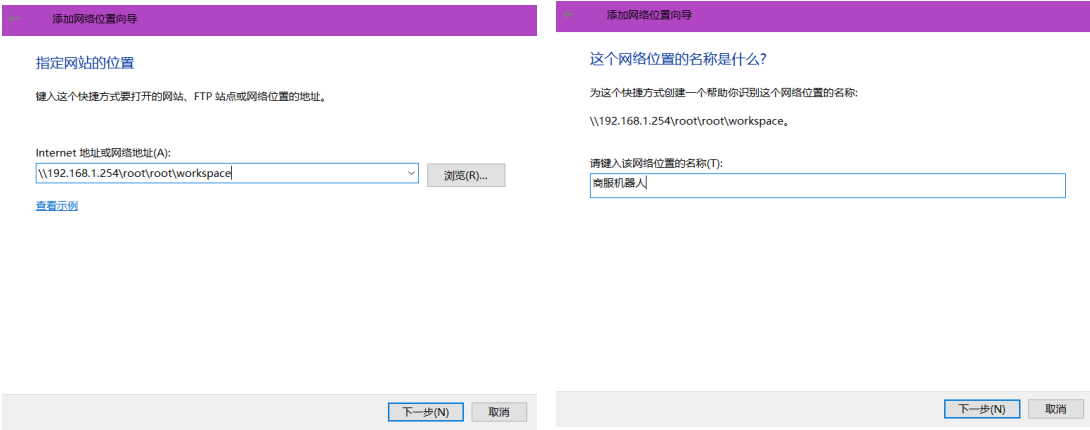


图 1-15 输入 IP 和命名网络位置

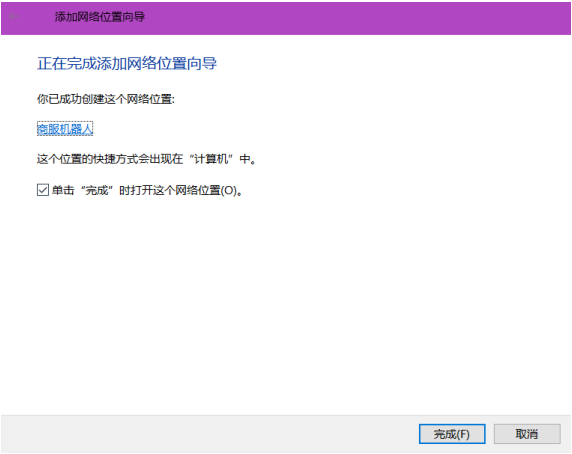


图 1-16 完成配置

第三步，设置完成后，会自动弹出文件夹（即为商服机器人上的工程文件夹），同时也可通过个人 PC 上的“此电脑”访问该文件夹，注意此路径仅可在已建立以太网连接下才可

以使用。

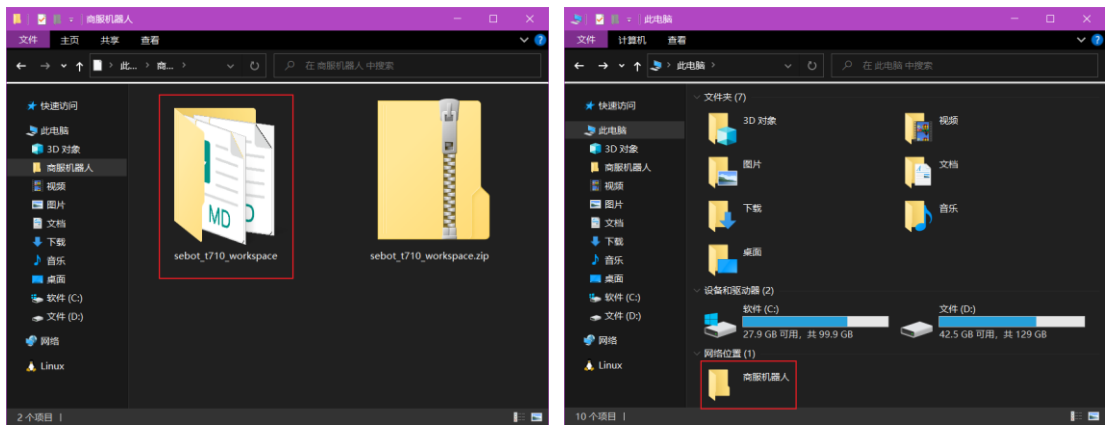


图 1-17 共享的文件夹

设置完成后，在网络位置下即可生成对应的文件夹路径，进入此文件夹“商服机器人”，打开路径 `sebot_t710_workspace`（商服机器人（机械臂版）为 `sebot-t710-competition`）文件夹内的 `sebot_ros_kits` 文件夹，即可查看商服机器人的所有工程文件。一般情况下我们通过 VSCode 查看此文件夹，此时通过个人电脑即可对商服机器人的文件资源进行增加、删除、修改、查询等操作。

步骤 3：查看工程文件

（1）打开方式选择，打开 VSCode

查看工程文件时，可以直接在机器人上通过 Files 打开，也可以将工程文件复制到个人电脑打开，还可以在个人 PC 与商服机器人建立远程连接的情况下打开。

在机器人本地打开工程文件时，直接在商服机器人的终端中输入以下命令即可打开 VSCode 编译工具。由于我们使用的是 `root` 用户，所以需要特殊的权限设置。

在机器人中打开终端，输入以下指令打开 VSCode。

```
code
root@paddlepi:~# code
[6573:1123/153309.885802:ERROR:viz_main_impl.cc(186)] Exiting GPU process due to
errors during initialization
[6599:1123/153310.111700:ERROR:gpu_memory_buffer_support_x11.cc(44)] dri3 extens
ion not supported.
[main 2023-11-23T07:33:10.226Z] update#setState idle
[main 2023-11-23T07:33:48.900Z] update#setState checking for updates
[main 2023-11-23T07:33:49.629Z] update#setState available for download
```

图 1-18 在终端中打开 VSCode

打开 VSCode 后，如果是开机后的第一次运行，需要输入密码，直接输入 `root` 即可。

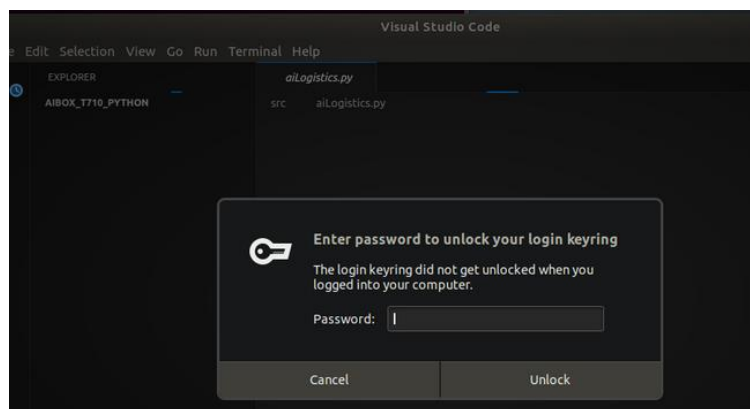


图 1-19 输入密码

如果是打开个人 PC 上已经复制好的工程文件或者在个人 PC 上远程访问机器人的工程文件，直接在本地电脑打开 VSCode 即可，无需输入上述指令。

(2) 打开工程文件

在个人 PC 或者机器人系统打开 VSCode 后，选择打开文件夹（File -- Open Folder），然后选择工程文件所在的目录，具体的路径直接参考下图（复制的工程文件，其路径自行打开）。

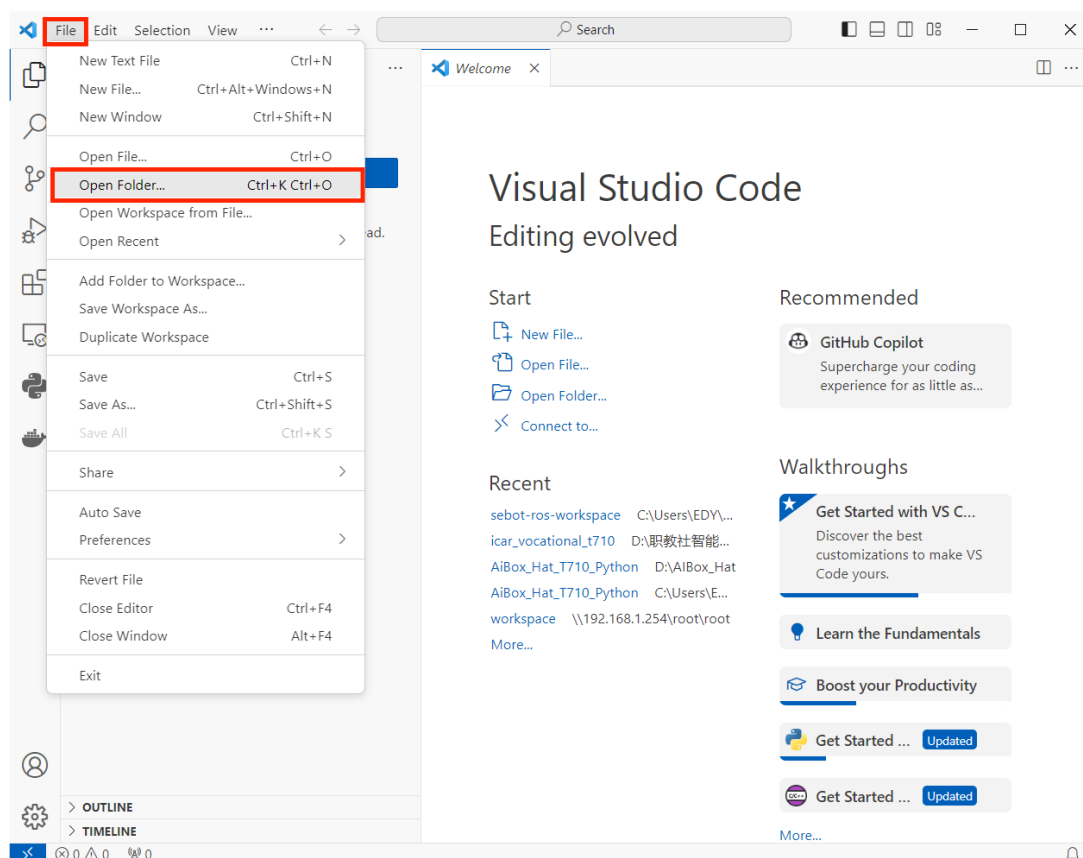


图 1-20 VSCode 打开工程文件夹

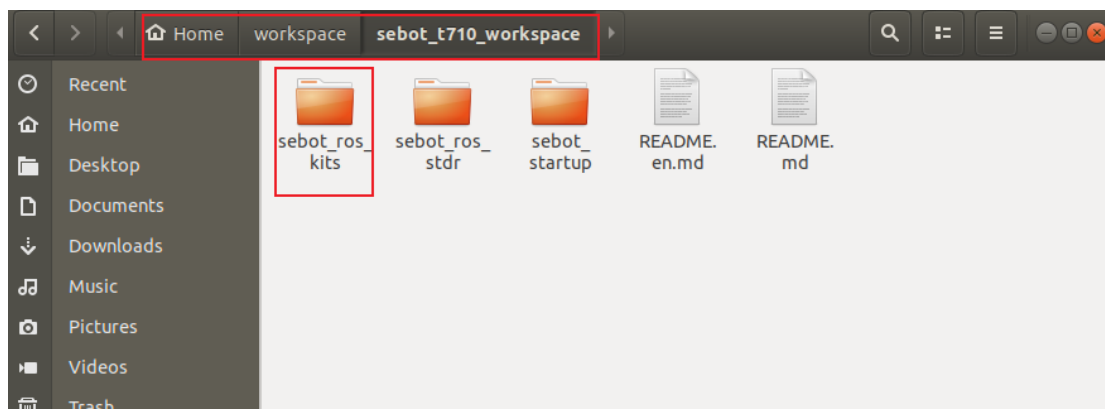


图 1-21 (a) 机器人系统路径

需要说明的是：商服机器人（机械臂版）的工程文件路径是 **workspace/sebot-t710-competition**，而非 **sebot_t710_workspace**，如下图示，商服机器人（机械臂版）的所有实训均基于 **sebot-t710-competition** 文件包进行，后续不再重复提醒。

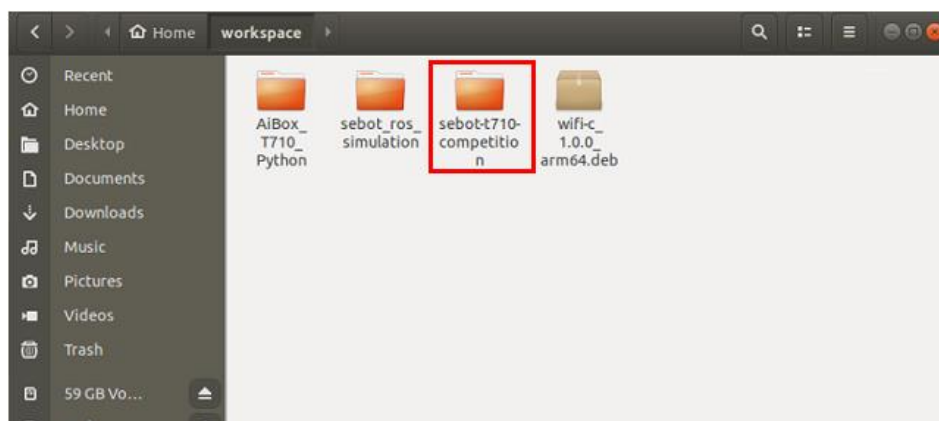


图 1-21 (b) 商服机器人（机械臂版）系统路径

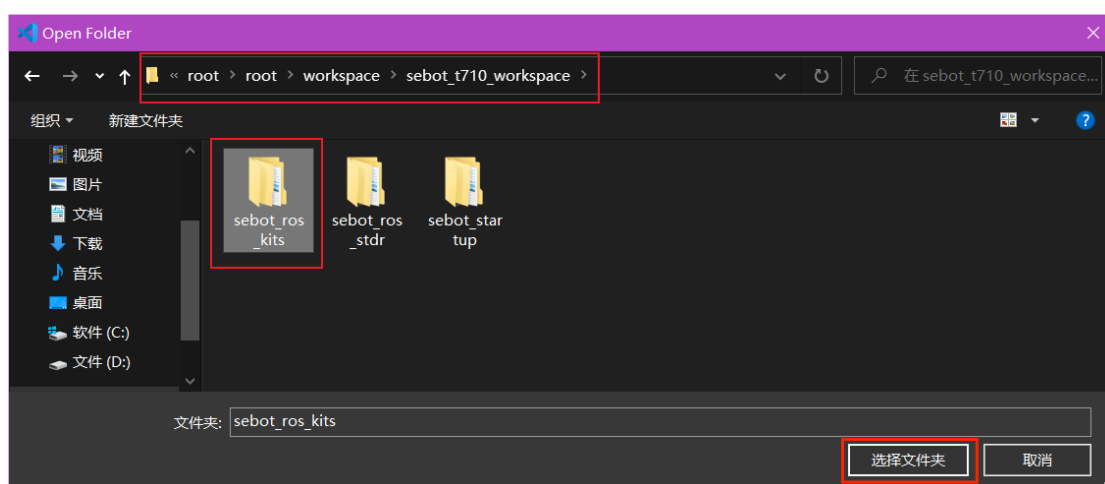


图 1-22 远程打开文件路径

打开文件夹后即可查看整个工程文件，可以在左侧查看相关文件，单击对应的文件即可打开查看文件内容并进行编辑操作。后续实验中查看和编辑文件内容时均与此步骤相同。

和“.launch”都是 xml 格式的文件，其作用主要是用于配置参数。在 ROS 系统中，“.launch”文件的主要作用就是可以启动多个节点，从而减少操作步骤。“.cpp”和“.py”则分别是使用 c++语言和 python 语言编写的程序源码，其代码体现了程序运行的逻辑过程。

需要编辑对应的文件时，直接单击对应的文件即可。

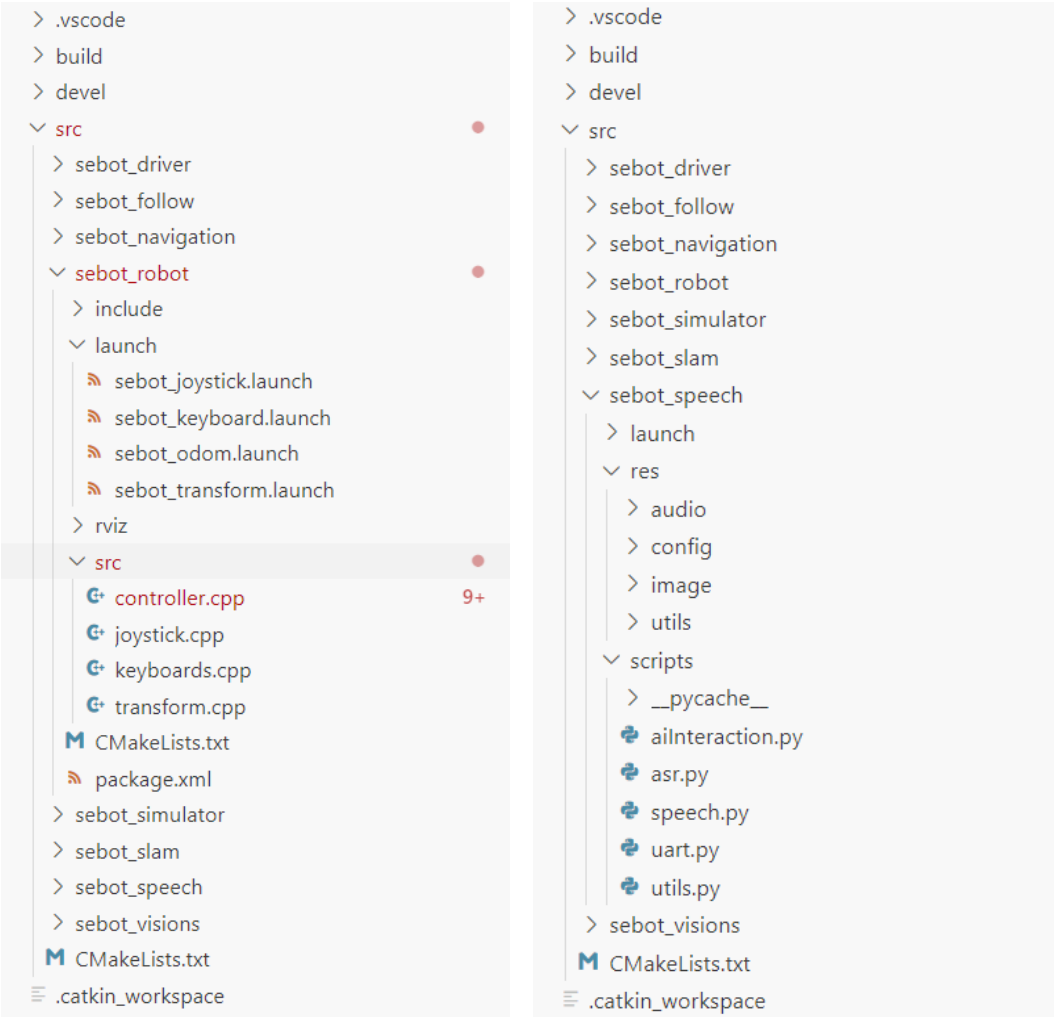


图 1-25 工程文件目录及各个类型的文件

步骤 4：创建自己的工作空间及功能包

上述步骤 3 中我们认识了本商服机器人已经创建完毕的工作空间及研发完成的工程文件，下面我们学习创建自己的工作空间及功能包方法。

商服机器人的操作系统是 ROS 系统，所以我们需要使用 ROS 系统的相关基础操作。开机启动机器人系统后，打开终端 Terminal，在终端中输入命令并按下回车即可执行对应的 ROS 命令。

(1) 创建工作空间

ROS 系统最基础的操作便是创建工作空间，工作空间是指一个包含 ROS 项目的顶级目录。它是一个文件夹，用于组织和存放 ROS 相关的代码、配置文件和编译生成的文件。

第一步，打开终端 **Terminal**，在终端中选择文件路径（一般使用根目录即可），然后输入以下命令创建工作空间文件夹以及内部存放功能包的 **src** 文件夹。

```
mkdir -p sebot_ws/src
```

如下操作中，**cd** 命令表示进入到指定的文件夹路径，**mkdir** 表示新建文件夹，**-p** 表示如果没有父目录则自动创建父目录，**sebot_ws** 即父目录，**src** 即子目录。文件夹路径、父目录及子目录都可以自行定义。

```
root@paddlepi:~# cd workspace/  
root@paddlepi:~/workspace# mkdir -p sebot_ws/src  
root@paddlepi:~/workspace#
```

图 1-26 创建文件夹命令

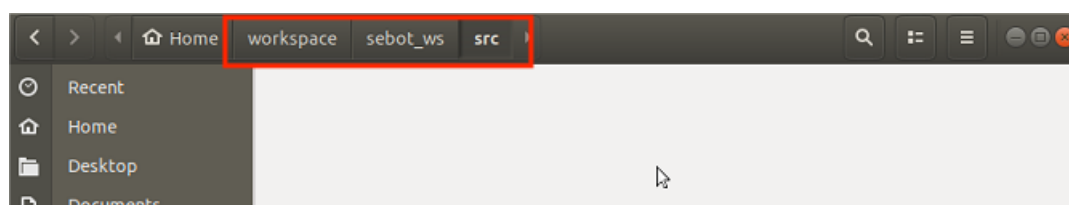


图 1-27 创建好的文件夹

第二步，进入到创建的文件夹 **sebot_ws/src/**中，执行以下命令初始化工作空间。

```
catkin_init_workspace
```

执行后可在 **src** 文件夹内看到生成了 **CMakeLists.txt** 文件，此文件是主要用来描述功能包在编译时的编译规则，即一些编译指令，使用的是 **CMake** 的语法。

```
root@paddlepi:~/workspace# cd sebot_ws/src/  
root@paddlepi:~/workspace/sebot_ws/src# catkin_init_workspace  
Creating symlink "/root/workspace/sebot_ws/src/CMakeLists.txt" pointing to "/opt/  
/ros/melodic/share/catkin/cmake/toplevel.cmake"  
root@paddlepi:~/workspace/sebot_ws/src#
```

图 1-28 工作空间初始化

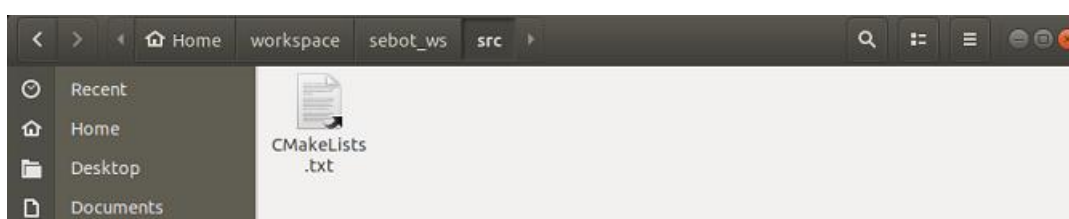


图 1-29 初始化完成

(2) 创建功能包

一个工作空间 `workspace` 可以包含多个功能包，功能包是 ROS 项目的基本单元，用于组织和管理相关功能的代码。功能包必须创建到工作空间的 `src` 文件目录下，创建时需要先进入到 `src` 文件目录，然后使用 `catkin_create_pkg` 命令，此命令后面跟着的参数表示具体的功能包名以及相关的依赖项，实际开发时根据需要自行编写。下面以最常使用的命令为例进行演示，具体命令如下，其中 `sebot_new` 即功能包名称，后面三个参数为依赖项。

```
catkin_create_pkg sebot_new std_msgs roscpp
```

```
root@paddlepi:~/workspace/sebot_ws/src# catkin_create_pkg sebot_new std_msgs roscpp
Created file sebot_new/package.xml
Created file sebot_new/CMakeLists.txt
Created folder sebot_new/include/sebot_new
Created folder sebot_new/src
Successfully created files in /root/workspace/sebot_ws/src/sebot_new. Please adjust the values in package.xml.
root@paddlepi:~/workspace/sebot_ws/src#
```

图 1-30 创建功能包

创建完成后即在 `src` 文件夹下生成了功能包 `sebot_new`，功能包内部自动生成了 `include` 和 `src` 文件夹，以及 `CMakeLists.txt` 和 `package.xml` 文件。`include` 文件夹主要用于存放 `c++` 头文件；`src` 则用于存放 `c++` 源码；`package.xml` 文件的开始主要是使用 `xml` 语言记录的功能包的名字、版本、描述和开源协议等信息，然后 `package.xml` 后面还包含功能包的依赖信息。在创建功能包时的 `std_msg/roscpp/rospy` 等依赖包都会写在这里面，后面如果还有附加的依赖功能包就可以在这里面手动加入；`CMakeLists.txt` 文件的作用前面已经说过。

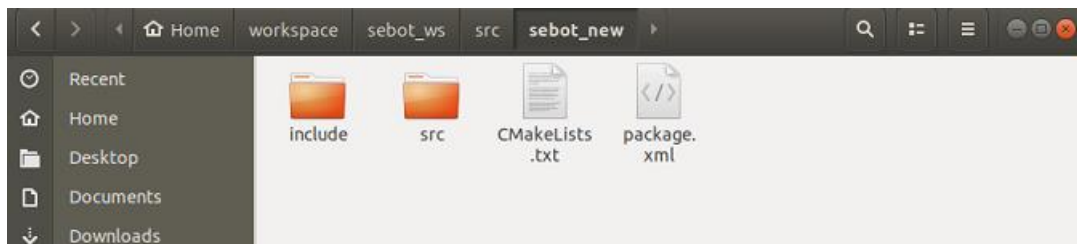


图 1-31 功能包创建完成

功能包创建完成后即可进行相关工程的编写，如果还需要其他的资源文件或者用 `python` 编写的脚本，则需要新建其他文件夹。

(3) 编译工程文件

工程文件编写完成后，需要根据实际情况选择是否编译。编译主要针对的是使用 `c++` 编写的源码，如果 `c++`（即 `.cpp` 文件）源码发生了改变并且需要运行相关节点，则必须经过编译这个步骤，其他的 `launch` 文件或者 `python` 脚本等发生了改变无需进行编译即可直接运行

相关节点。

编译时，需要进入到工作空间的根目录 `sebot_ws`，然后执行以下编译命令，执行后等待进度完成重新出现文件路径提示即编译完成。其他工作空间的编译与此同理。

```
catkin_make -l

root@paddlepi:~/workspace# cd sebot_ws/
root@paddlepi:~/workspace/sebot_ws# catkin_make -l
```

图 1-32 编译命令

```
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Failed
-- Looking for pthread_create in pthreads
-- Looking for pthread_create in pthreads - not found
-- Looking for pthread_create in pthread
-- Looking for pthread_create in pthread - found
-- Found Threads: TRUE
-- Using Python nosetests: /usr/bin/nosetests-2.7
-- catkin 0.7.29
-- BUILD_SHARED_LIBS is on
-- BUILD_SHARED_LIBS is on
-- 
-- traversing 1 packages in topological order:
--   - sebot_new
-- 
-- +++ processing catkin package: 'sebot_new'
-- ==> add_subdirectory(sebot_new)
-- Configuring done
-- Generating done
-- Build files have been written to: /root/workspace/sebot_ws/build
####
#### Running command: "make -l" in "/root/workspace/sebot_ws/build"
####
root@paddlepi:~/workspace/sebot_ws#
```

图 1-33 编译完成

编译完成后，工作空间中会生成 `devel`、`build` 这两个文件夹，`devel` 为开发空间，用于放置开发过程中的可执行文件；`build` 文件夹用于存放编译过程中产生的中间文件。

编译后需要把工作空间添加到环境变量中去，可以使用下面的指令，把工作空间 `devel` 文件夹下的 `setup.bash` 文件的路径加到环境变量中。

```
source devel/setup.bash
```

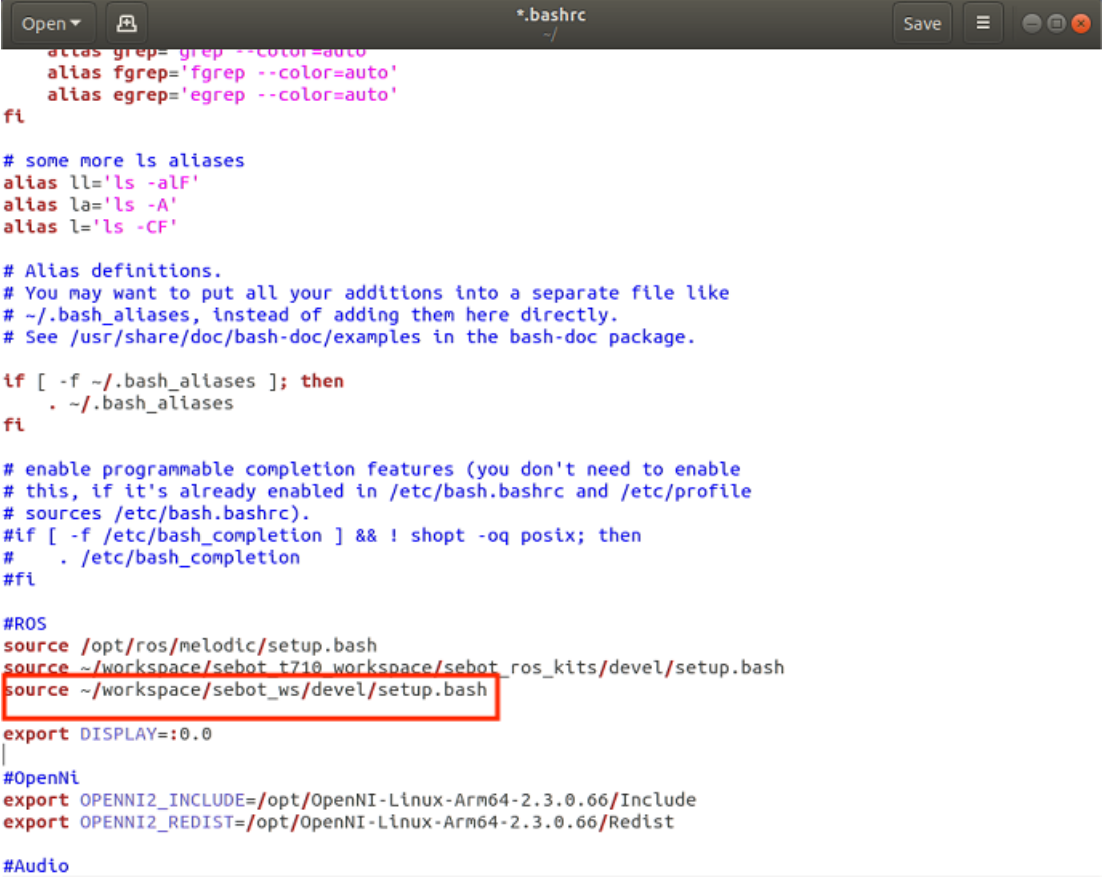
这样做不方便的一点是每次运行都得重新 `source` 一下，我们可以把路径直接加到 `~/.bashrc` 文件中。`bashrc` 文件是 `home` 目录下的一个 `shell` 文件，用于储存用户的个性化设置。在系统每次启动时都会加载 `bashrc` 文件中的内容，并根据内容定制当前 `bash` 的配置和环境。输入以下命令打开 `~/.bashrc` 文件。

```
gedit ~/.bashrc
```

选取空行输入以下指令添加环境变量。

```
source ~/workspace/sebot_ws/devel/setup.bash
```

使用快捷键“ctrl”+“s”保存后退出即可。



```
Open ▾  *.bashrc  Save  ≡  ⌵  ✖
alias grep='grep --color=auto'
alias fgrep='fgrep --color=auto'
alias egrep='egrep --color=auto'
fi

# some more ls aliases
alias ll='ls -alF'
alias la='ls -A'
alias l='ls -CF'

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
#if [ -f /etc/bash_completion ] && ! shopt -oq posix; then
#    . /etc/bash_completion
#fi

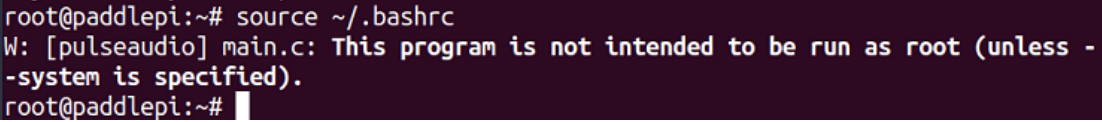
#ROS
source /opt/ros/melodic/setup.bash
source ~/workspace/sebot_ws/devel/setup.bash
source ~/workspace/sebot_ws/devel/setup.bash
export DISPLAY=:0.0
|
#OpenNI
export OPENNII2_INCLUDE=/opt/OpenNI-Linux-Arm64-2.3.0.66/Include
export OPENNII2_REDIST=/opt/OpenNI-Linux-Arm64-2.3.0.66/Redist

#Audio
```

图 1-34 添加环境变量

保存完成后在终端使用以下指令运行一遍 `bashrc` 文件确保环境变量起效，执行后出现路径提示即表示运行完成。

```
source ~/.bashrc
```



```
root@paddlepi:~# source ~/.bashrc
W: [pulseaudio] main.c: This program is not intended to be run as root (unless -
-system is specified).
root@paddlepi:~#
```

图 1-35 运行完成

添加环境变量时需要注意操作正确，并且不可修改其他内容，否则可能导致其他问题。

步骤 5：运行节点，通过手柄或键盘控制机器人移动

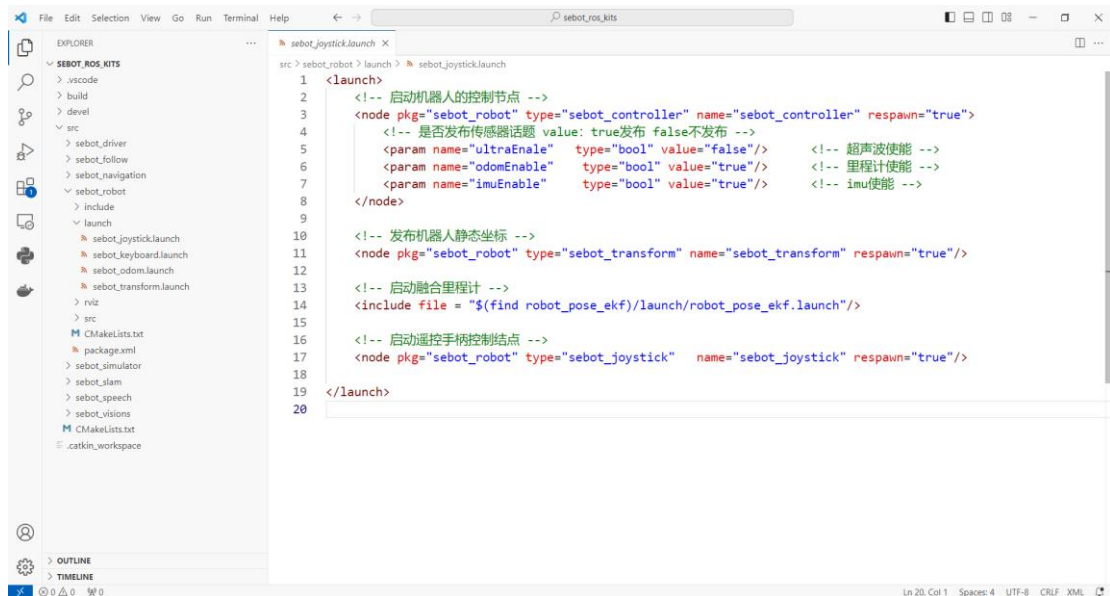
下面我们基于已经开发完成的商服机器人工程文件进行运行节点的操作，其逻辑与我们前面步骤 4 进行自定义工作空间的操作是相同的，如果要运行自定义的节点可参考此步骤的

操作。

ROS 运行节点即运行功能包内的可执行文件。在 ROS 中，功能包是用于组织和包含 ROS 相关代码和文件的基本单元。但是，ROS 并不能直接运行整个功能包。功能包本身并不包含可执行文件，而只是提供了一些用于构建和配置 ROS 系统的文件和工具。如果要运行 ROS 节点，你需要编译包含该节点的功能包，并将生成的可执行文件与 ROS 系统中的其他节点进行通信。

运行节点时，可以运行单个节点也可以依次启动多个节点。运行单个节点时，需要先在终端中使用命令 `roscore` 启动节点管理器，然后在新终端中使用 `roslaunch` 命令启动功能包中的可执行文件。在实际使用中这种方法较少，因为一个完整的功能效果往往是由多个节点相互协作完成的，因此下面详细说明启动多个节点的命令。

ROS 中启动多个节点的命令是 `roslaunch`，此命令可以自动根据相关的配置文件启动相关节点，`launch` 文件也是功能包中的组成文件，其作用是作为配置文件可以同时启动多个节点并且配置相关节点的参数。下图中 `sebot_robot/launch` 即 `launch` 文件所在的路径。



```
1 <!-- 启动机器人的控制节点 -->
2 <!-- 是否发布传感器话题 value: true发布 false不发布 -->
3 <node pkg="sebot_robot" type="sebot_controller" name="sebot_controller" respawn="true">
4   <param name="ultraEnable" type="bool" value="false"/> <!-- 超声波使能 -->
5   <param name="odomEnable" type="bool" value="true"/> <!-- 里程计使能 -->
6   <param name="imuEnable" type="bool" value="true"/> <!-- imu使能 -->
7 </node>
8
9
10 <!-- 发布机器人静态坐标 -->
11 <node pkg="sebot_robot" type="sebot_transform" name="sebot_transform" respawn="true"/>
12
13 <!-- 启动融合里程计 -->
14 <include file = "$(find robot_pose_ekf)/launch/robot_pose_ekf.launch"/>
15
16 <!-- 启动遥控手柄控制节点 -->
17 <node pkg="sebot_robot" type="sebot_joystick" name="sebot_joystick" respawn="true"/>
18
19 </launch>
20
```

图 1-36 launch 文件内容

`sebot_robot/src` 文件夹即源码文件夹（我们可以自定义），商服机器人控制的相关源码均在此文件夹内，其中 `joystick.cpp` 为手柄控制逻辑的源码，`keyboards.cpp` 为键盘控制逻辑的源码，`controller.cpp` 为机器人整体控制的源码，相关源码逻辑可结合程序注释自行理解。

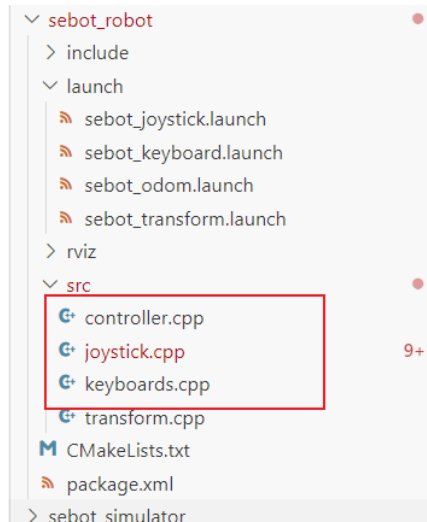


图 1-37 商服机器人的源码文件

运行节点时，直接在终端中输入对应的 `roslaunch` 命令即可，此命令后跟随两个参数，第一个参数是功能包名，第二个参数是具体的 `launch` 文件名。

以控制机器人为例，商服机器人提供了两种控制方式，分别是手柄控制和键盘控制。输入对应的命令后按下回车，等待程序运行后在对应的终端中使用手柄或键盘即可实现手柄控制或键盘控制。如果需要停止对应的命令，则在对应的终端中按下“`ctrl`”+“`c`”终止当前进程，如果需要开启新终端，可使用快捷键“`ctrl`”+“`alt`”+“`t`”。

手柄控制节点命令如下：

```
roslaunch sebot_robot sebot_joystick.launch
```

```

/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_robot/launch/sebot_joystick.launch http://localhost:11311
File Edit View Search Terminal Help
robot_pose_ekf (robot_pose_ekf/robot_pose_ekf)
sebot_controller (sebot_robot/sebot_controller)
sebot_joystick (sebot_robot/sebot_joystick)
sebot_transform (sebot_robot/sebot_transform)

auto-starting new master
process[master]: started with pid [21731]
ROS_MASTER_URI=http://localhost:11311

setting /run_id to 431a4fcc-82c3-11ee-aa8e-00606edd3042
process[rosout-1]: started with pid [21752]
started core service [/rosout]
process[sebot_controller-2]: started with pid [21759]
process[sebot_transform-3]: started with pid [21767]
process[robot_pose_ekf-4]: started with pid [21772]
process[sebot_joystick-5]: started with pid [21785]
[ INFO] [1699948572.418620479]: output frame: odom_combined
[ INFO] [1699948572.432088140]: base frame: base_footprint
[ INFO] [1699948572.748415789]: Initializing Odom sensor
[ INFO] [1699948572.796042868]: Initializing Imu sensor
[ INFO] [1699948572.796961245]: Odom sensor activated
[ INFO] [1699948572.797174858]: Imu sensor activated
[ INFO] [1699948572.808783727]: Kalman filter initialized with odom measurement

```

图 1-38 手柄遥控节点运行后的终端界面

手柄控制时，按下手柄上的按键即可执行指令，松开则停止。



图 1-39 手柄控制按键说明

键盘控制节点命令如下：

```
roslaunch sebot_robot sebot_keyboard.launch
```

```

/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_robot/launch/sebot_keyboard.launch http://localhost:11311
File Edit View Search Terminal Help
=====
PARAMETERS
* /rostdistro: melodic
* /rosversion: 1.14.13
* /sebot_controller/imuEnable: False
* /sebot_controller/odomEnable: False
* /sebot_controller/ultraEnale: False
I
NODES
/
  sebot_controller (sebot_robot/sebot_controller)
  sebot_keyboard (sebot_robot/sebot_keyboard)

auto-starting new master
process[master]: started with pid [22876]
ROS_MASTER_URI=http://localhost:11311

setting /run_id to 9155a042-82c3-11ee-bfec-00606edd3042
process[rosout-1]: started with pid [22897]
started core service [/rosout]
process[sebot_controller-2]: started with pid [22905]
process[sebot_keyboard-3]: started with pid [22913]

```

图 1-40 键盘控制节点运行后的终端界面

键盘控制时，按下按键会持续执行指令，按下空格才能停止执行。

使用键盘控制机器人时的按键使用说明如下图，使用时如果按下无反应，注意先检查是否唤醒对应的控制设备以及电量是否充足。

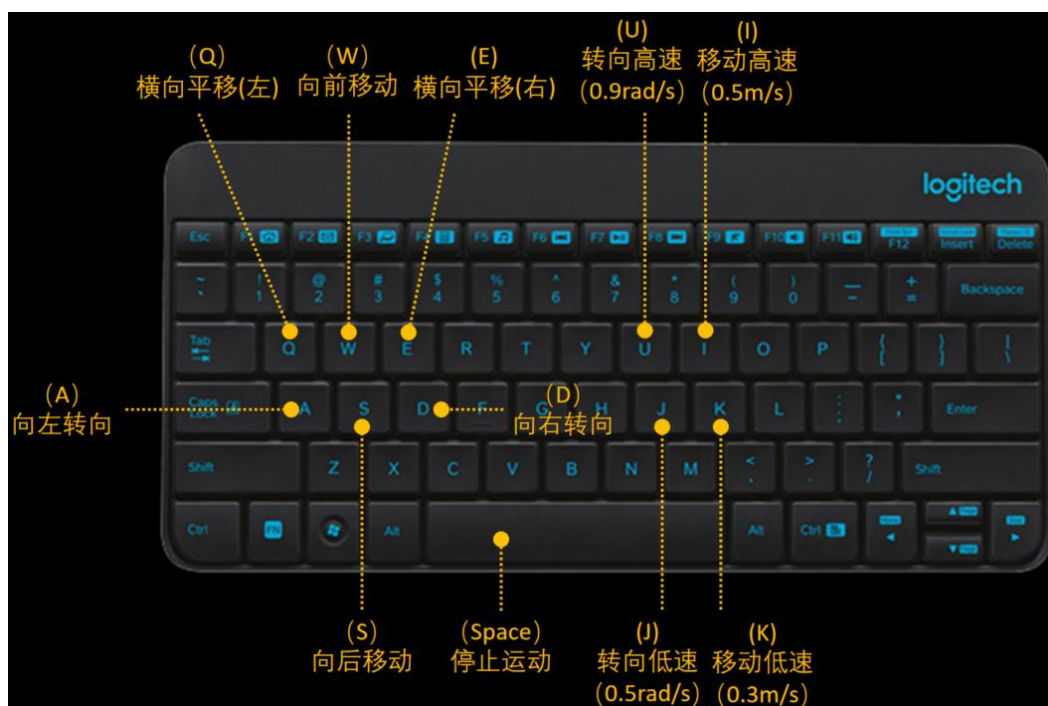


图 1-41 键盘控制按键说明

开启相应节点后，同学们即可尝试通过手柄或键盘控制机器人进行前进、后退、左转、右转、高速移动、低速移动等操作了，观察机器人移动情况，后续实验中需要操控机器人进行运动时即需要采用本步骤。

步骤 6：话题的发布与接收操作

话题是 ROS 中用于消息传递的机制。通过话题可以实现节点间的通信，以此实现节点间的协作。在 ROS 中，执行对应的节点功能后，可以在命令行中输入对应的命令从而查看话题相关的内容。

下面以手柄遥控为例，查看其话题的相关内容。

(1) 查看话题列表及话题内容

运行手柄遥控的 launch 文件后（`roslaunch sebot_robot sebot_joystick.launch`），开启一个新终端，在终端中输入以下命令即可查看当前所有的话题列表。

```
rostopic list
```

```
root@paddlepi:~# rostopic list
/cmd_vel
/imu
/odom
/odom_combined
/rosout
/rosout_agg
/tf
/tf_static
root@paddlepi:~#
```

图 1-42 查看话题列表

如上图示，通信中有/cmd_vel 、/imu、/odom、/tf 等话题列表。

获得话题列表后，输入以下命令即可查看/cmd_vel 话题的发布者和接收者。

```
rostopic info /cmd_vel
```

Publishers 即发布者，负责向话题发送消息，Subscribers 即订阅者，负责从话题内接收数据。

```
^Croot@paddlepi:~# rostopic info /cmd_vel
Type: geometry_msgs/Twist

Publishers:
* /sebot_joystick (http://paddlepi:43633/)

Subscribers:
* /sebot_controller (http://paddlepi:39573/)
* /rostopic_23261_1699952700550 (http://paddlepi:37841/)
* /rostopic_23846_1699952756388 (http://paddlepi:34861/)

root@paddlepi:~#
```

图 1-43 查看话题发布者和订阅者

通过类似命令可以分别查看/imu、/odom、/tf 等话题的发布者及订阅者。

(2) 订阅并接收话题消息

在终端中输入以下命令可以订阅话题/cmd_vel 并显示其内容。

```
rostopic echo /cmd_vel
```

此话题内的数据为按下遥控手柄的信息，因此在按下手柄的按键后，终端才会出现相关的数据，否则是没有数据的。

接收的消息是对机器人底盘的控制，linear 即线速度，angular 即角速度，x、y、z 表示

的是三维坐标轴。当机器人发生移动或旋转时，对应的数据会发生变化。

```
root@paddlepi:~# rostopic echo /cmd_vel
linear:
  x: 0.300000011921
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
```

图 1-44 订阅并输出话题信息

(3) 发布话题消息

运行手柄遥控程序时，节点间会自行在话题中发布和接收数据，同时我们也可以通过在终端中输入指令来手动发布话题数据。在终端中输入以下指令即可在/cmd_vel 话题中发布使机器人旋转的控制信息。其中 rostopic pub -r 10 是默认指令，/cmd_vel 是话题名称，geometry_msgs/Twist 是数据类型和格式，linear 即线速度，angular 即角速度，x、y、z 表示的是三维坐标轴。

```
rostopic pub -r 10 /cmd_vel geometry_msgs/Twist '{linear: {x: 0, y: 0, z: 0}, angular: {x: 0, y: 0, z: 0.1}}'
```

指令执行后/cmd_vel 话题内会重复发出对应的消息，在新终端中订阅并接收/cmd_vel 话题消息即可看到消息内容。订阅节点并接收到消息内容后便会控制机器人原地转圈，按下“ctrl”+“c”即可停止在此话题发布数据。由于话题数据已经被接收，因此需要覆盖数据才能停止机器人旋转，所以在停止消息发布的终端后，切换到手柄控制的终端并按下手柄上的任意一个方向键并松开即可覆盖数据使机器人停止转动。

```
root@paddlepi:~# rostopic pub -r 10 /cmd_vel geometry_msgs/Twist '{linear: {x: 0, y: 0, z: 0}, angular: {x: 0, y: 0, z: 0.1}}'
```

图 1-45 向话题内发布数据

人的使用，所使用仿真工具主要是 RViz，在后续将进一步详细介绍。

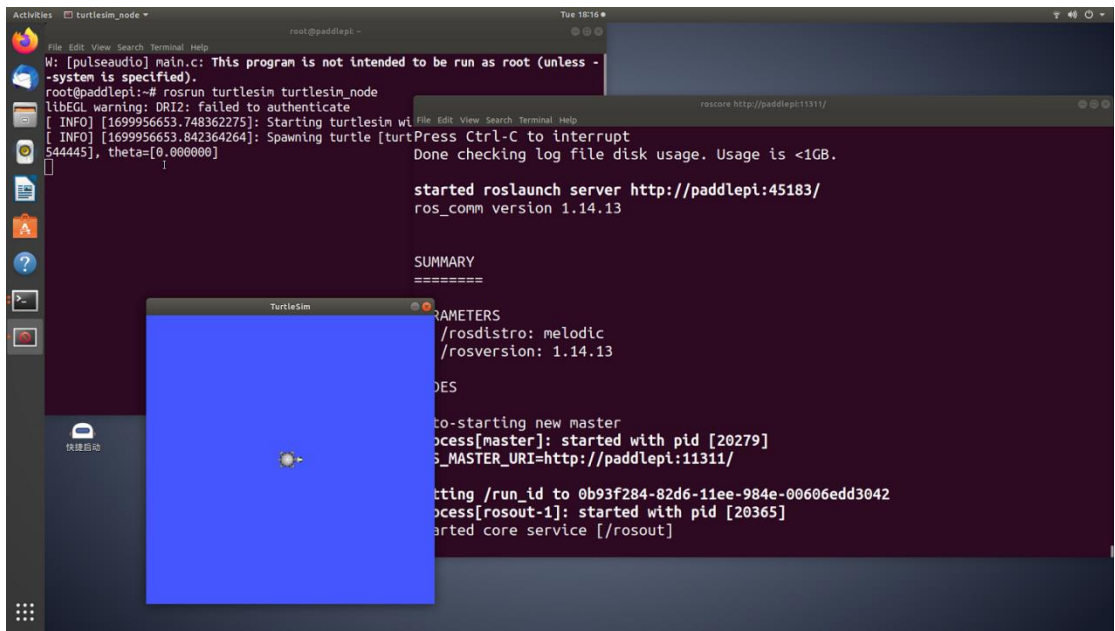


图 1-47 运行小海龟仿真器